

CS691 • SPRINT 0

AgentMesh

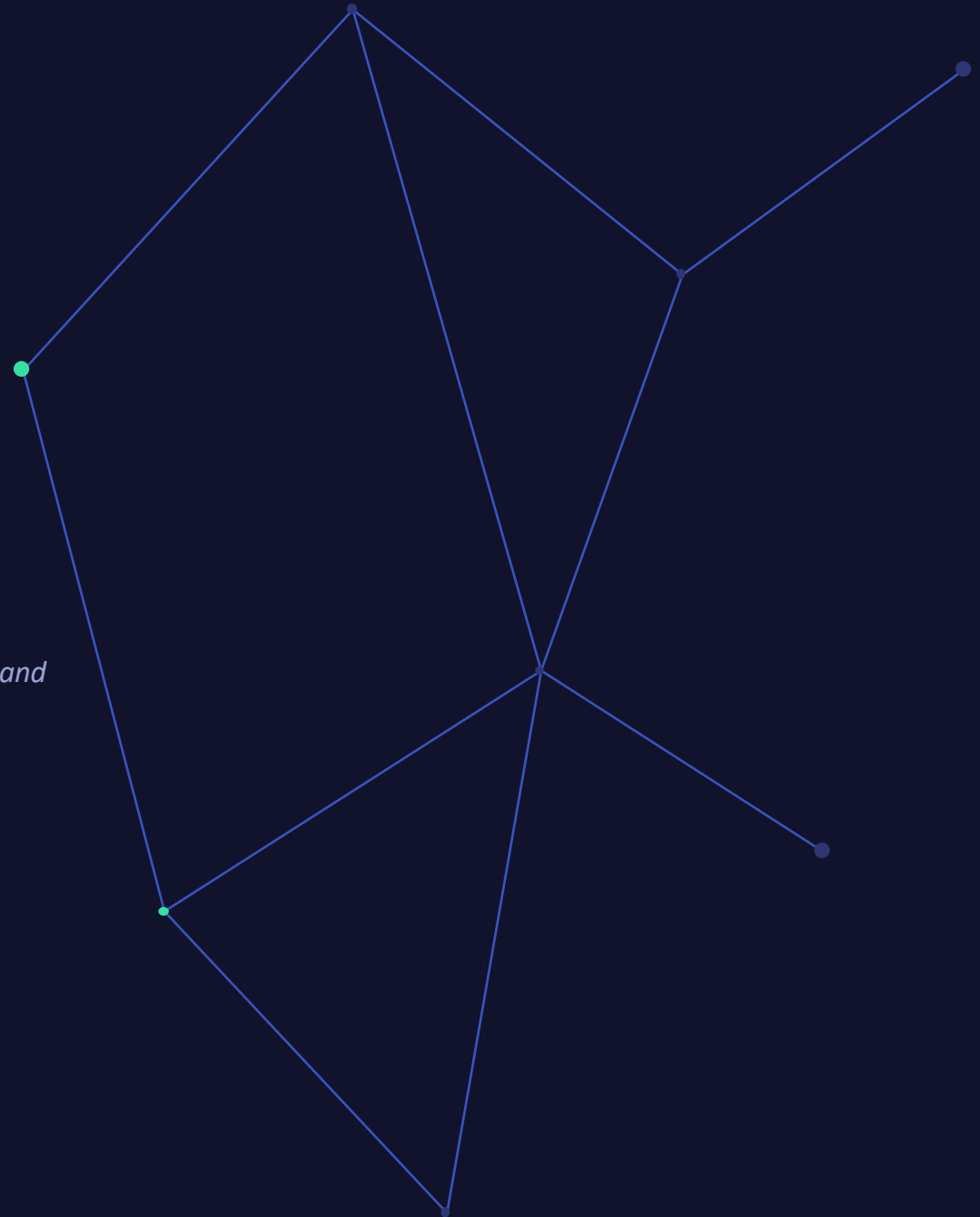
An AI Agent Orchestration Platform

Coordinating specialized AI agents and multiple LLM providers through sequential, parallel, and conditional workflows.

Team Members: 7 • Group Presentation — Sprint Review

Team 2 - Phoenix

Sprint 0 • September 2026



What We'll Cover

1

Team Roles & Responsibilities

Who's doing what across the 7-member team

3

Project Description

What AgentMesh is and how it works

5

Technologies & Algorithms

The stack and logic behind orchestration

7

Team Working Agreement

How we hold each other accountable

2

Problem Statement

The gap we're solving, and our hypothesis

4

Personas

Three user types AgentMesh is built for

6

Project Schedule

Meeting cadence and Sprint deliverables

8

Retrospective

Sprint 0 reflection and next actions

Team Member Roles & Responsibilities

1. Dhwani Dobariya – Team Leader/ Documentation
2. Aniruddha Rath - Backend Engineer
3. Puneet Tulsiani - Frontend Engineer
4. Soumen Nageshkar - Full-Stack / Platform Engineer
5. Sutej Kulkarni - DevOps / Infrastructure Engineer
6. Gurjot Singh - QA / Test Engineer
7. Om Jadhav - Documentation & Agile Lead

Why General-Purpose AI Isn't Enough

The Problem

- A single model isn't optimal for research, coding, security, testing, and analysis all at once.
- Running every available AI agent on every request wastes time and API tokens.
- Complex tasks have real dependencies between subtasks that a flat chat model can't express.
- Passing full conversations between steps creates unnecessary, costly context.
- Users rarely know why a given AI capability was invoked, or what it actually did.

HYPOTHESIS

If a central Orchestrator selects only the specialized AI agents a request actually needs, and coordinates them through sequential, parallel, and conditional workflows,

then complex multi-step tasks will complete with lower cost, lower latency, and higher reliability than sending the entire task to one general-purpose model.

What AgentMesh Is

AgentMesh is an AI Agent Orchestration Platform that coordinates multiple specialized AI agents and multiple LLM providers to solve complex tasks — acting as an intelligent workflow engine for AI, rather than a single chatbot.



Who We're Building AgentMesh For



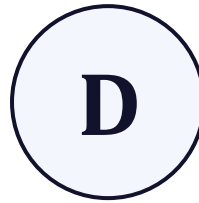
Priya, Software Engineering Lead

GOAL

Wants a single pipeline that runs security review, code fixes, and tests automatically before merge.

FRUSTRATION

Manually chaining separate AI tools for security, coding, and testing is slow and error-prone.



Daniel, Data Analyst

GOAL

Needs multi-step analysis — cleaning, analysis, and reporting — done from one plain-language request.

FRUSTRATION

General chat models lose track of steps and can't hand structured data cleanly between stages.



Aisha, Technical Product Manager

GOAL

Wants visibility into which AI capability ran, why, and what it cost, for every workflow.

FRUSTRATION

Black-box chatbot answers make it impossible to audit cost, latency, or decision quality.

Proposed Technology Stack

F**Frontend**

- React / Next.js / TypeScript / CSS3 / Framer
- Workflow visualization UI

B**Backend**

- Python (FastAPI) / SQLAlchemy / Celery

D**Data & Caching**

- PostgreSQL / MongoDB
- Redis (queue + cache)

A**AI / ML**

- OpenAI, Anthropic, Gemini, others

I**Infra & Auth**

- Docker containers
- JWT / OAuth

Q**Quality**

- Unit, integration & E2E tests
- Structured logging & metrics

Logic Behind the Orchestrator

Capability-Matching / Classification

1

Maps each request to only the agent capabilities it actually needs.

Conditional Branching Logic

3

Chooses the next step based on intermediate results (e.g., vulnerability severity).

Retry / Fallback Strategy

5

Retries, swaps providers, or escalates to a human on repeated failure.

Dependency Graph Resolution

2

Orders tasks (topological sort) so dependent steps wait on prerequisites.

Cost-Aware Routing

4

Selects the cheapest capable model per task; reserves strong models for hard steps.

Schema Validation

6

Checks structured agent outputs against an expected schema before they move on.

How the Team Meets & Delivers ?

Meeting Cadence

- Full team stand-up: 2x per week (status + blockers)
- Async check-ins on team channel between meetings
- Sprint planning at the start of every Sprint
- Sprint review + retrospective at the end of every Sprint

Sprint 0 Deliverables

- Sprint 0 presentation (slides + recorded video + PDF)
- Group wiki page with project overview & team info
- Team working agreement (Word + PDF)
- Sprint 0 retrospective, recorded
- Github code

How We Work Together

When & Where

Stand-ups 2x/week on a shared call; async updates posted daily in the team channel.

Deadlines

All individual work is due 48 hours before a Sprint review — no same-day drops on one person.

Reminders

Teammates give one friendly reminder before a deadline; after that, it's on the individual.

Commitment

If you can't finish a task, flag it in the channel at least 24 hours before the deadline.

Shared Load

Reviews and integration are shared across the team, not owned by a single 'fixer.'

Conflict

Disagreements are raised in the team meeting, not worked around silently.

Sprint 0 Retrospective

What Went Well

Solid Architecture: Clean modular design, Docker setup, and backend framework.

Agent Workflows: Effective orchestration tools, messaging layers, and workflow engines.

Testing: Comprehensive automated tests ensuring code stability

What Can Improve

Documentation: Need clearer setup instructions and architecture guides.

Debugging Complexity: Monitoring and tracing multi-agent execution paths need optimization.

Action Items

Enhance Monitoring: Add better real-time metrics for tracking agent executions.

Speed Up Tests: Optimize the test suite for faster CI/CD pipelines.

Group Wikipage

<https://github.com/htmwf2026-T2-Phoenix/wiki>

Thank you — questions welcome.

